HW3

Environment Setup

Component	Details
OS	Debian GNU/Linux, Kernel 5.10.0-34-cloud-amd64
CPU	Intel Xeon @ 2.20GHz — 1 socket, 2 cores, 2 threads/core (4 logical CPUs)
Memory	15 GiB RAM
GPU	NVIDIA L4 — 23034 MiB VRAM, TDP 72W
Driver	NVIDIA 550.90.07
CUDA	12.4 (nvcc V12.4.131, built Mar 28 2024)

Part-A Report

Q1 — `vecadd00`

Values/Thread	Vector Size	Time (s)	GFlops/s	GBytes/s
500	3,840,000	0.000324	11.85	142.22
1000	7,680,000	0.000880	8.73	104.73
2000	15,360,000	0.001855	8.28	99.37

- As values-per-thread doubles (500→1000→2000), kernel time roughly doubles (0.000324s → 0.000880s → 0.001855s), consistent with a linear increase in work per thread.
- **Throughput drops** as problem size grows: GFlops/s falls from 11.85 → 8.73 → 8.28, suggesting memory bandwidth becomes the bottleneck at larger sizes.
- Peak bandwidth achieved is only ~142 GB/s at the smallest size, well below theoretical GPU memory bandwidth, indicating **non-coalesced access** in `vecaddKernel00` is limiting performance.

Q2 — `vecadd01` vs `vecadd00`

Values/Thread	vecadd00 (s)	vecadd01 (s)	Speedup	BW00 (GB/s)	BW01 (GB/s)
500	0.000324	0.000102	**3.18×**	142.22	451.57
1000	0.000880	0.000533	**1.65×**	104.73	172.87
2000	0.001855	0.001064	**1.74×**	99.37	173.26

- `vecadd01` is consistently faster across all sizes, with the most dramatic improvement at size 500 (**3.18× speedup**, bandwidth 142 → 452 GB/s).
- The improvement is directly attributable to **coalesced memory access**: when threads in a warp read contiguous addresses simultaneously, the GPU hardware can serve the entire warp in a single memory transaction rather than multiple serialised ones.

- At larger sizes (1000, 2000), the speedup narrows to ~1.65–1.74×. This suggests that at larger working sets, other factors (e.g. L2 cache pressure, warp scheduling latency) begin to limit absolute throughput, but coalescing still provides a clear and consistent advantage.

Q3: Baseline Performance (`matmultKernel00` , FOOTPRINT = 16)

Parameter	Matrix Size	Grid	Block	Time (sec)	GFlop/s
16	256×256	16×16	16×16	0.000027	1,245.5
32	512×512	32×32	16×16	0.000128	2,096.6
64	1024×1024	64×64	16×16	0.000911	2,357.3

Performance scales with matrix size because larger matrices expose more parallelism and better amortize kernel launch overhead. The GPU's occupancy improves as more thread blocks are available to hide memory latency.

Q4: Optimized Performance (`matmultKernel01` , FOOTPRINT = 32)

Each thread now computes a **2×2 sub-tile** of C (4 values), doubling the footprint per dimension.

Parameter	Matrix Size	Grid	Block	Time (sec)	GFlop/s
8	256×256	8×8	16×16	0.000022	1,529.8
16	512×512	16×16	16×16	0.000059	4,539.9
32	1024×1024	32×32	16×16	0.000343	6,259.3

Speedup Over Baseline

Matrix Size	`matmult00` (GFlop/s)	`matmult01` (GFlop/s)	Speedup
256×256	1,245.5	1,529.8	**1.23×**
512×512	2,096.6	4,539.9	**2.17×**
1024×1024	2,357.3	6,259.3	**2.66×**

The optimized kernel achieves up to **2.66× speedup** on 1024×1024 matrices. The speedup increases with matrix size because larger problems better saturate the GPU and amortize the overhead of the optimization.

Why the Optimization Works

1. **Higher compute-to-memory ratio**: Each thread loads 2× more data from global memory into shared memory , but computes 4× more output values (a 2×2 sub-tile of C). Since computation grows faster than memory traffic, the ratio of arithmetic operations to memory accesses doubles, reducing the memory bottleneck.
2. **Coalesced memory access**: Threads in a warp access consecutive global memory addresses, ensuring coalesced reads for both `shared_A` and `shared_B`.

Q5: Rules of Thumb for CUDA Performance

1. **Maximize the compute-to-memory ratio.** Have each thread compute multiple output values from the same shared memory tile. This reduces redundant global memory traffic, which is the primary bottleneck.
2. **Ensure coalesced global memory access.** Adjacent threads (consecutive `threadIdx.x`) should access adjacent memory locations. Strided or scattered access patterns waste bandwidth significantly.
3. **Scale the problem to saturate the GPU.** Small matrices underutilize the hardware. Performance (GFlop/s) consistently improves with problem size because more thread blocks are available to hide latency and fill the GPU's compute units.
4. **Balance block size with occupancy.** A 16×16 thread block (256 threads) is a good default — it's large enough for high occupancy but doesn't consume too many registers or shared memory per block.

Part-B Report

Q1 CPU Only

K (millions)	Array Size	Time (ms)
1	1,000,000	0.000
5	5,000,000	0.003
10	10,000,000	0.006
50	50,000,000	0.034
100	100,000,000	0.067

Q2 CUDA without Unified Memory

K	Array Size	Scenario 1 (sec)	Scenario 2 (sec)	Scenario 3 (sec)
1	1,000,000	0.038358	0.000611	0.000015
5	5,000,000	0.316169	0.007258	0.000242
10	10,000,000	0.635488	0.014519	0.000484
50	50,000,000	3.188103	0.073053	0.002555
100	100,000,000	6.333274	0.144441	0.005114

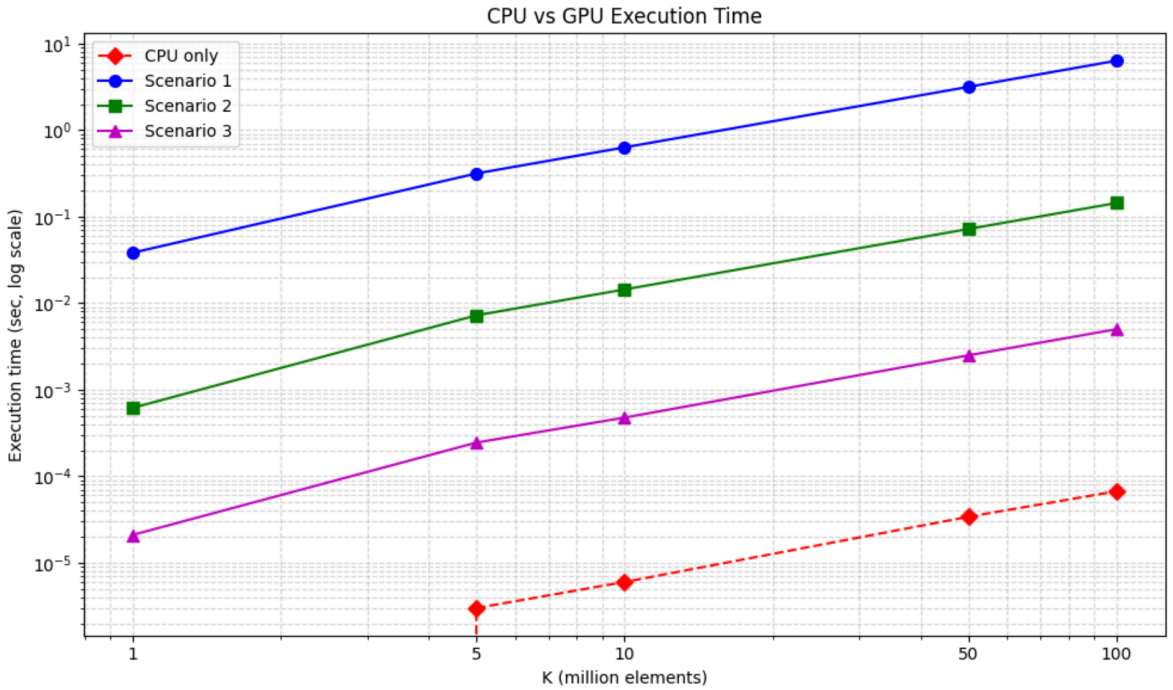
Q3 CUDA with Unified Memory

K	Array Size	Scenario 1 (sec)	Scenario 2 (sec)	Scenario 3 (sec)
1	1,000,000	0.038265	0.000617	0.000021
5	5,000,000	0.315325	0.007236	0.000245

K	Array Size	Scenario 1 (sec)	Scenario 2 (sec)	Scenario 3 (sec)
10	10,000,000	0.631571	0.014406	0.000476
50	50,000,000	3.154956	0.071942	0.002495
100	100,000,000	6.334772	0.143587	0.004998

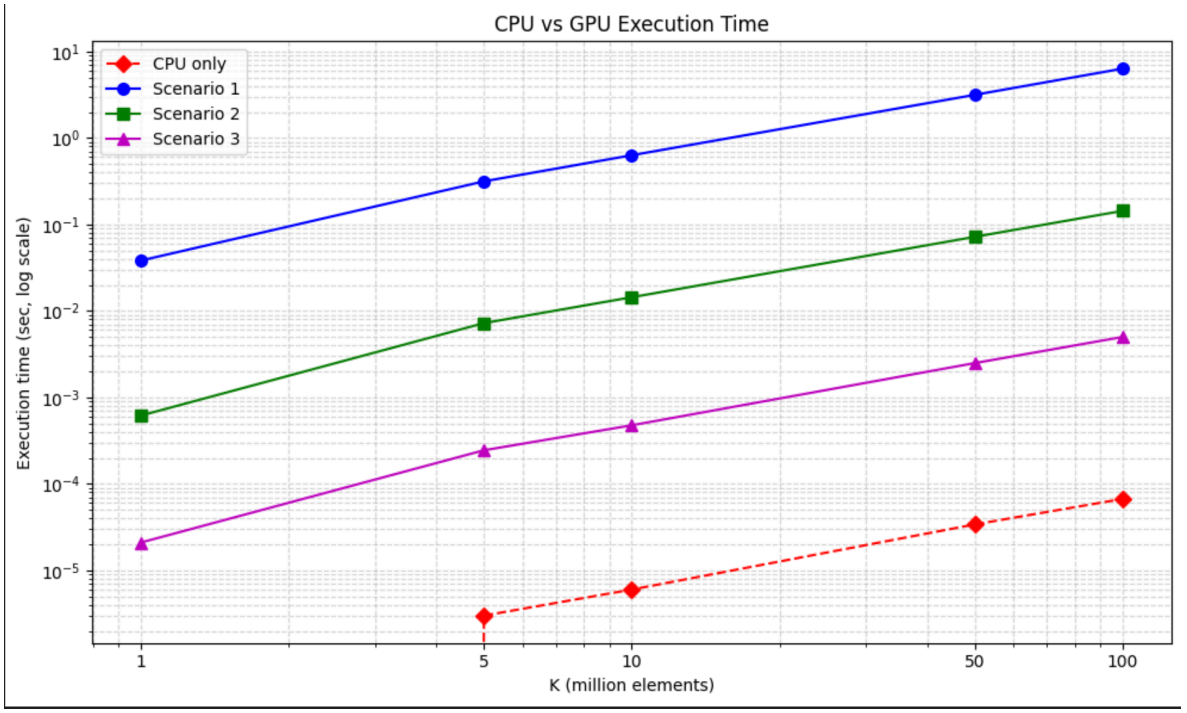
Q4

CUDA without Unified Memory v.s. CPU Only



- The log-log chart shows all four lines clearly separated.
- CPU only (dashed red) sits at the bottom since the CPU times are in microseconds,
- while Scenario 1 (blue) is the slowest at several seconds for large K.
- Scenario 2 (green) and Scenario 3 (purple) fall in between, with Scenario 3 consistently the fastest GPU approach.

CUDA with Unified Memory v.s. CPU Only



Q2 (explicit `cudaMemcpy`) and Q3 (`cudaMallocManaged`) show nearly identical kernel execution times. Although explicit transfers should theoretically be slightly faster, the simple sequential access pattern of array addition allows Unified Memory to prefetch pages efficiently, making the difference negligible.

Part-C Report

C1 Convolution in CUDA

Kernel	Time (ms)	Checksum
c1	8.453	122756344698240.0

C2 Tiled Convolution with CUDA

Kernel	Time (ms)	Checksum
c2	8.000	122756344698240.0

- Compared to c2, c1 uses shared memory and tiling, but the performance improvement is marginal (8.453 ms vs 8.000 ms).
- This is likely because the overhead of loading data into shared memory offsets the benefit of reduced global memory accesses, especially when the computation is not memory-bound enough to benefit significantly from tiling.

C3 Convolution with cuDNN

Kernel	Time (ms)	Checksum
c3 (cuDNN)	19.773	122756344698240.0

cuDNN is slower here (19.773 ms vs ~8 ms) likely due to:

1. **FP64 precision**: cuDNN is heavily optimized for FP32/FP16 and can leverage Tensor Cores for these precisions. With `float64`, it falls back to a generic compute path, losing most optimizations.
2. **Runtime overhead**: cuDNN's internal algorithm selection and workspace allocation add non-trivial overhead for a relatively small convolution problem.

C4 Convolution with OpenAI Triton

	Time (ms)	Checksum
c4	7.994	122756344698240.0

Screen Shot

Part-A

Q1

```
========== Part-A Q1: vecadd00 (500, 1000, 2000) ==========
--- vecadd00 500 ---
Total vector size: 3840000
Time: 0.000324 (sec), GFlopsS: 11.851455, GBytesS: 142.217460
Test PASSED

real    0m1.789s
user    0m0.053s
sys     0m1.730s

--- vecadd00 1000 ---
Total vector size: 7680000
Time: 0.000880 (sec), GFlopsS: 8.727243, GBytesS: 104.726919
Test PASSED

real    0m1.795s
user    0m0.065s
sys     0m1.724s

--- vecadd00 2000 ---
Total vector size: 15360000
Time: 0.001855 (sec), GFlopsS: 8.280785, GBytesS: 99.369423
Test PASSED

real    0m1.858s
user    0m0.113s
sys     0m1.742s
```

Q2

```
========== Part-A Q2: vecadd01 (500, 1000, 2000) ==========
--- vecadd01 500 ---
Total vector size: 3840000
Time: 0.000102 (sec), GFlopsS: 37.631139, GBytesS: 451.573664
Test PASSED

real    0m1.755s
user    0m0.052s
sys     0m1.700s

--- vecadd01 1000 ---
Total vector size: 7680000
Time: 0.000533 (sec), GFlopsS: 14.406196, GBytesS: 172.874354
Test PASSED

real    0m1.797s
user    0m0.052s
sys     0m1.743s

--- vecadd01 2000 ---
Total vector size: 15360000
Time: 0.001064 (sec), GFlopsS: 14.438483, GBytesS: 173.261791
Test PASSED

real    0m1.870s
user    0m0.109s
sys     0m1.753s
```

Q3

```
========== Part-A Q3: matmult00 (16, 32, 64) ==========
--- matmult00 16 ---
Data dimensions: 256x256
Grid Dimensions: 16x16
Block Dimensions: 16x16
Footprint Dimensions: 16x16
Time: 0.000027 (sec), nFlops: 33554432, GFlopsS: 1245.464499

real    0m1.730s
user    0m0.021s
sys     0m1.705s

--- matmult00 32 ---
Data dimensions: 512x512
Grid Dimensions: 32x32
Block Dimensions: 16x16
Footprint Dimensions: 16x16
Time: 0.000128 (sec), nFlops: 268435456, GFlopsS: 2096.647871

real    0m1.739s
user    0m0.021s
sys     0m1.714s

--- matmult00 64 ---
Data dimensions: 1024x1024
Grid Dimensions: 64x64
Block Dimensions: 16x16
Footprint Dimensions: 16x16
Time: 0.000911 (sec), nFlops: 2147483648, GFlopsS: 2357.288473

real    0m1.749s
user    0m0.021s
sys     0m1.722s
```

Q4

```
========== Part-A Q4: matmult01 (8, 16, 32) ==========
--- matmult01 8 ---
Data dimensions: 256x256
Grid Dimensions: 8x8
Block Dimensions: 16x16
Footprint Dimensions: 32x32
Time: 0.000022 (sec), nFlops: 33554432, GFlopsS: 1529.755308

real    0m1.770s
user    0m0.045s
sys     0m1.716s

--- matmult01 16 ---
Data dimensions: 512x512
Grid Dimensions: 16x16
Block Dimensions: 16x16
Footprint Dimensions: 32x32
Time: 0.000059 (sec), nFlops: 268435456, GFlopsS: 4539.918979

real    0m1.753s
user    0m0.028s
sys     0m1.722s

--- matmult01 32 ---
Data dimensions: 1024x1024
Grid Dimensions: 32x32
Block Dimensions: 16x16
Footprint Dimensions: 32x32
Time: 0.000343 (sec), nFlops: 2147483648, GFlopsS: 6259.346251

real    0m1.804s
user    0m0.029s
sys     0m1.767s
```

Part-B

Q1

```
========== Part-B Q1: CPU only (K = 1, 5, 10, 50, 100) ==========
--- q1 K=1 ---
Time taken: 0 ms

real    0m0.008s
user    0m0.000s
sys     0m0.008s

--- q1 K=5 ---
Time taken: 0.003 ms

real    0m0.023s
user    0m0.005s
sys     0m0.018s

--- q1 K=10 ---
Time taken: 0.006 ms

real    0m0.041s
user    0m0.011s
sys     0m0.030s

--- q1 K=50 ---
Time taken: 0.034 ms

real    0m0.199s
user    0m0.093s
sys     0m0.105s

--- q1 K=100 ---
Time taken: 0.067 ms

real    0m0.387s
user    0m0.185s
sys     0m0.201s
```

Q2

```
========== Part-B Q2: GPU without unified (K × scenario 1,2,3) ==========
--- q2 K=1 scenario=1 ---
Total vector size: 1000000
Time: 0.038358 (sec)
Test PASSED

real    0m1.887s
user    0m0.097s
sys     0m1.782s

--- q2 K=1 scenario=2 ---
Total vector size: 1000000
Time: 0.000611 (sec)
Test PASSED

real    0m1.759s
user    0m0.016s
sys     0m1.739s

--- q2 K=1 scenario=3 ---
Total vector size: 1000000
Time: 0.000015 (sec)
Test PASSED

real    0m1.750s
user    0m0.032s
sys     0m1.710s

--- q2 K=5 scenario=1 ---
Total vector size: 5000000
Time: 0.316169 (sec)
Test PASSED

real    0m2.357s
user    0m0.628s
sys     0m1.724s

--- q2 K=5 scenario=2 ---
Total vector size: 5000000
Time: 0.007258 (sec)
Test PASSED

real    0m1.788s
user    0m0.065s
sys     0m1.718s

--- q2 K=5 scenario=3 ---
Total vector size: 5000000
Time: 0.000242 (sec)
Test PASSED

real    0m1.781s
user    0m0.049s
sys     0m1.724s
```

```
--- q2 K=10 scenario=1 ---
Total vector size: 10000000
Time: 0.635488 (sec)
Test PASSED

real    0m3.091s
user    0m1.346s
sys     0m1.741s

--- q2 K=10 scenario=2 ---
Total vector size: 10000000
Time: 0.014519 (sec)
Test PASSED

real    0m1.865s
user    0m0.089s
sys     0m1.768s

--- q2 K=10 scenario=3 ---
Total vector size: 10000000
Time: 0.000484 (sec)
Test PASSED

real    0m1.821s
user    0m0.073s
sys     0m1.740s

--- q2 K=50 scenario=1 ---
Total vector size: 50000000
Time: 3.188103 (sec)
Test PASSED

real    0m8.505s
user    0m6.678s
sys     0m1.820s

--- q2 K=50 scenario=2 ---
Total vector size: 50000000
Time: 0.073053 (sec)
Test PASSED

real    0m2.277s
user    0m0.409s
sys     0m1.858s

--- q2 K=50 scenario=3 ---
Total vector size: 50000000
Time: 0.002555 (sec)
Test PASSED

real    0m2.136s
user    0m0.283s
sys     0m1.850s
```

```
--- q2 K=100 scenario=1 ---
Total vector size: 100000000
Time: 6.333274 (sec)
Test PASSED

real    0m15.200s
user    0m13.176s
sys     0m2.023s

--- q2 K=100 scenario=2 ---
Total vector size: 100000000
Time: 0.144441 (sec)
Test PASSED

real    0m2.804s
user    0m0.806s
sys     0m1.994s

--- q2 K=100 scenario=3 ---
Total vector size: 100000000
Time: 0.005114 (sec)
Test PASSED

real    0m2.517s
user    0m0.481s
sys     0m2.031s
```

Q3

```
========== Part-B Q3: GPU with unified (K × scenario 1,2,3) ==========
--- q3 K=1 scenario=1 ---
Total vector size: 1000000
Time: 0.038265 (sec)
Test PASSED

real    0m1.826s
user    0m0.105s
sys     0m1.714s

--- q3 K=1 scenario=2 ---
Total vector size: 1000000
Time: 0.000617 (sec)
Test PASSED

real    0m1.746s
user    0m0.028s
sys     0m1.713s

--- q3 K=1 scenario=3 ---
Total vector size: 1000000
Time: 0.000021 (sec)
Test PASSED

real    0m1.760s
user    0m0.012s
sys     0m1.735s

--- q3 K=5 scenario=1 ---
Total vector size: 5000000
Time: 0.315325 (sec)
Test PASSED

real    0m2.302s
user    0m0.561s
sys     0m1.732s

--- q3 K=5 scenario=2 ---
Total vector size: 5000000
Time: 0.007236 (sec)
Test PASSED

real    0m1.797s
user    0m0.057s
sys     0m1.736s

--- q3 K=5 scenario=3 ---
Total vector size: 5000000
Time: 0.000245 (sec)
Test PASSED

real    0m1.781s
user    0m0.041s
sys     0m1.735s
```

```
--- q3 K=10 scenario=1 ---
Total vector size: 10000000
Time: 0.631571 (sec)
Test PASSED

real    0m2.861s
user    0m1.105s
sys     0m1.754s

--- q3 K=10 scenario=2 ---
Total vector size: 10000000
Time: 0.014406 (sec)
Test PASSED

real    0m1.858s
user    0m0.088s
sys     0m1.760s

--- q3 K=10 scenario=3 ---
Total vector size: 10000000
Time: 0.000476 (sec)
Test PASSED

real    0m1.816s
user    0m0.073s
sys     0m1.741s

--- q3 K=50 scenario=1 ---
Total vector size: 50000000
Time: 3.154956 (sec)
Test PASSED

real    0m7.389s
user    0m5.457s
sys     0m1.930s

--- q3 K=50 scenario=2 ---
Total vector size: 50000000
Time: 0.071942 (sec)
Test PASSED

real    0m2.315s
user    0m0.401s
sys     0m1.903s

--- q3 K=50 scenario=3 ---
Total vector size: 50000000
Time: 0.002495 (sec)
Test PASSED

real    0m2.161s
user    0m0.263s
sys     0m1.893s
```

```
--- q3 K=100 scenario=1 ---
Total vector size: 100000000
Time: 6.334772 (sec)
Test PASSED

real    0m13.106s
user    0m11.014s
sys     0m2.092s

--- q3 K=100 scenario=2 ---
Total vector size: 100000000
Time: 0.143587 (sec)
Test PASSED

real    0m2.897s
user    0m0.785s
sys     0m2.110s

--- q3 K=100 scenario=3 ---
Total vector size: 100000000
Time: 0.004998 (sec)
Test PASSED

real    0m2.613s
user    0m0.505s
sys     0m2.104s
```

Part-C

C1 ~ C3

```
========== Part-C: c1, c2, c3 ==========
--- c1 ---
Time: 8.453 ms
Checksum: 122756344698240.000000

real    0m2.108s
user    0m0.238s
sys     0m1.863s

--- c2 ---
Time: 8.000 ms
Checksum: 122756344698240.000000

real    0m2.107s
user    0m0.247s
sys     0m1.859s

--- c3 ---
Selected algorithm: 1, time: 20.377 ms
cuDNN convolution time: 19.773 ms
Checksum: 122756344698240.000000

real    0m2.403s
user    0m0.324s
sys     0m1.912s
```

C4

```python
# Testing

# Comparing the result from my_conv2d and Conv from torch
my_output, execution_time = my_conv2d(tensor_I, tensor_F)

torch.testing.assert_close(golden_out, my_output) # Assert statement should be passed
# Printing the execution time
print(f"Execution Time for triton kernel: {execution_time:.3f} ms")
```

[149] Python

```
Execution Time for triton kernel: 7.994 ms
```